

UNREAL ENGINE MULTIPLAYER PROJECT

Parcel Knight

Steam 리슨 서버에서 택배 상자를 운반하고 목표 구역에 배송해 팀 점수를 쌓는 멀티플레이 게임

팀명: Parcel Knight

발표자: 한수현

발표 날짜: 2026.07.24

한수현 — 게임 룰, 상점, 아이템

김로운 — 캐릭터

조영웅 — 택배 시스템, 캐릭터, 전체 UI

김경수 — 전체 택배 시스템

양석현 — 맵, 함정, UI



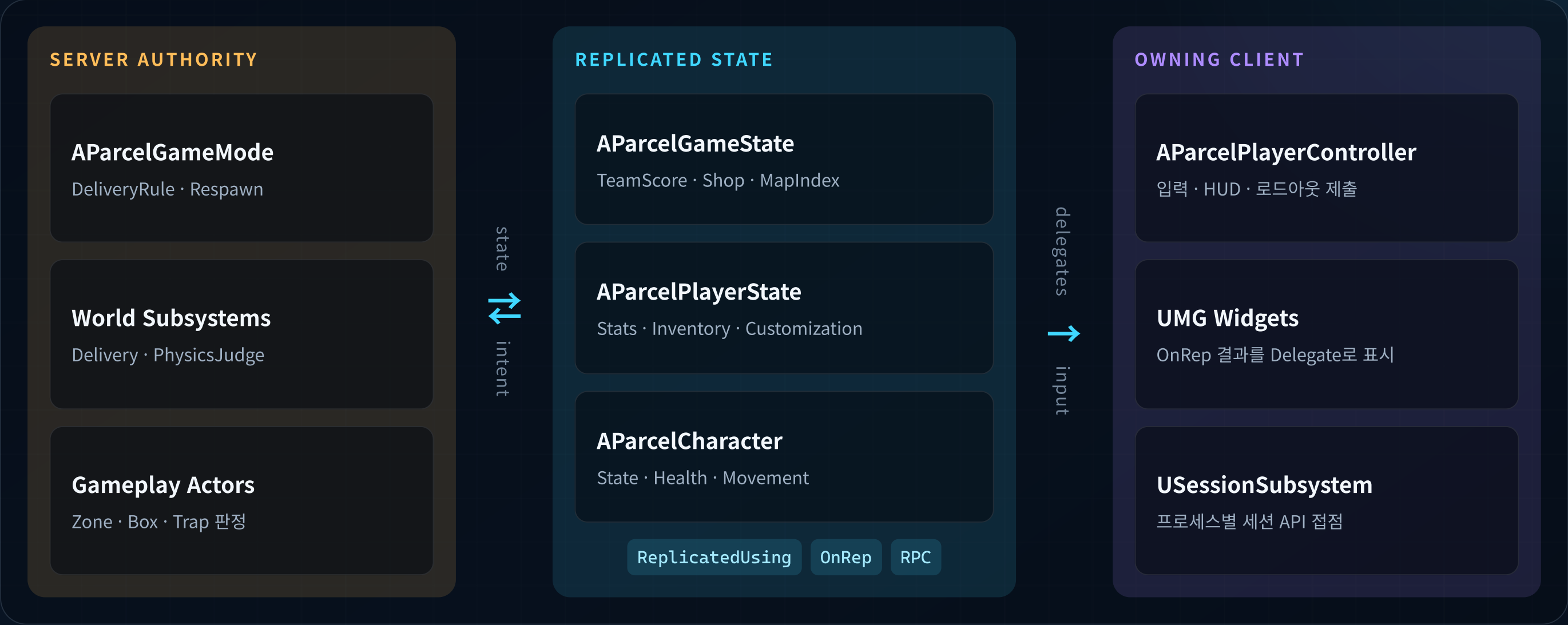
PROJECT OVERVIEW

게임 개요와 코드로 확인한 플레이 루프



SYSTEM MAP

전체 기술 구조 — 권한과 복제의 경계



CORE FEATURES · 3

기존 배송 플레이를 확장한 세 가지 핵심 기능

01

인벤토리 및 상점

로컬 상점 구매·저장과 스테이지 서버 인벤토리 복제를 서로 다른 책임으로 운영

다음: `GameInstance + InventoryComponent`

02

배송 파이프라인 유지

상자 생성·운반·배송 판정의 기존 서버 흐름을 건드리지 않고 플레이어 기능을 연동

다음: `Subsystem + Interface + Zone`

03

상자 물리 충돌

바닥에서는 Pawn과 충돌하고, 운반 중에는 끄며, 드롭 뒤 물리 상태를 복원

다음: `DeliveryBox + CarryComponent`

FEATURE 01

인벤토리 및 상점 — 저장과 네트워크 상태를 분리

- 01 RequestPurchase(ItemId) — 상점 항목 선택
- 02 TryPurchaseConsumable — 태그·중복·가격·잔액 확인
- 03 Money 차감 + OwnedConsumables 등록 + SaveData
- 04 RefreshShopUI — 상점·보유 목록·요약 재구축
- 05 스테이지 진입 시 Server_SubmitLoadout → 복제

플레이어 경험

구매 직후 보유 목록과 재화가 갱신되고, 선택한 로드아웃이 스테이지에서 사용됨

역할 경계

구매는 로컬 SaveGame. 서버는 제출된 장착 목록의 개수·태그·중복·카탈로그를 검증

서버 권한 소유 프로파일은 없음. 로컬 SaveGame은 조작 방지 수단이 아님.

TryPurchaseConsumable

ParcelGameInstance.cpp

```
if (ItemData→Price < 0) return false;  
if (HasOwnedConsumable(ItemTag)) return false;  
if (Money < ItemData→Price) return false;
```

```
Money -= ItemData→Price;  
OwnedConsumables.Add(ItemTag);  
SaveData();
```

FItemData는 ItemTag·Price·Effects를 정의. 스테이지의
UInventoryComponent::Items는 RepNotify로 UI에 전달.

FEATURE 02

기존 배송 파이프라인을 유지하면서 기능을 연동

ADeliveryZone::ProcessDelivery

DeliveryZone.cpp

```
if (!Box || !HasAuthority()) return;
if (Box->HasStateTag(FGameplayTag::RequestGameplayTag(
    TEXT("Box.State.Delivered"))) ||
    Box->HasStateTag(FGameplayTag::RequestGameplayTag(
    TEXT("Box.State.Failed"))))
{
    return;
}
```

```
FBoxData Data = Box->GetBoxData();
bool bIsCorrectZone =
    Data.TargetZoneTag.MatchesTagExact(ZoneTag);
```

성공·실패 상태와 목적지 판정은 계속 ADeliveryZone의 서버 경계 안에 있음

01 ADeliveryBoxSpawner timer → UDeliverySubsystem::SpawnRandomBox

02 InitializeBox → Box.State.Spawned

03 IInteractableInterface → ICarryableInterface

04 UCharacterCarryComponent Attach / Detach

05 Zone overlap → GameMode → DeliveryRule → Score → Despawn

유지한 경계

상점·인벤토리는 BoxStateTags와 배송
성공/실패를 직접 변경하지 않음

네트워크 역할

서버가 스폰·상태·판정. BoxData·상태
·HoldingCarrier는 복제되어 표현에 사용

StageData·Box DataTable·ZoneTag의 실제 Blueprint 할당은 [에디터 확인 필요]

FEATURE 03

DeliveryBox와 캐릭터의 물리 충돌 처리

이전 · 프리셋 의존

- 상자 Root는 UBoxComponent
- PhysicsBody 프로파일과 Simulate Physics만 설정
- Pawn 응답과 Collision Enabled를 코드에서 명시하지 않음
- Character Blueprint의 캡슐 설정과 조합에 따라 결과가 달라질 여지

목적

바닥의 상자가 캐릭터를 통과하지 않도록 양쪽 채널을 대칭 설정

바닥: QueryAndPhysics



운반: NoCollision



드롭: 물리·충돌 복원

현재 · 코드로 명시

```
// DeliveryBox  
CollisionComponent→SetCollisionEnabled(  
    ECollisionEnabled::QueryAndPhysics);  
CollisionComponent→SetCollisionResponseToChannel(  
    ECC_Pawn, ECR_Block);
```

```
// ParcelCharacter Capsule  
GetCapsuleComponent()→  
>SetCollisionResponseToChannel(  
    ECC_PhysicsBody, ECR_Block);
```

상태 전이

Attach 때 Actor collision/physics를 끄고, Detach 뒤
QueryAndPhysics·simulation·hit notify를 복원

네트워크

상자 이동 복제 활성화

OnRep_CarriedBox가 각 머신의 Attach/Detach 충돌 상태 적용

Blueprint preset [에디터 확인 필요]

원인 파악 → 검증 → 해결 기준

문제	확인된 원인	해결 방향	상태
01 호스트 네임플레이트	IsLocallyControlled() 로컬 숨김 분기가 표시를 차단	호스트·클라이언트 화면에서 네임플레이트 정상 표시	해결 완료
02 C++ / Blueprint 노드	현재 Editor DLL보다 UFUNCTION 소스가 새로워 기존 BP 노드가 해석되지 않음	Editor Target 재빌드 후 노드 갱신·컴파일	작업 필요
03 Box / Character 충돌	프로필 의존으로 Pawn·PhysicsBody 응답이 명시되지 않았음	양쪽 Block + 운반 상태별 비활성/복원	정적 확인

해결 결과 호스트 화면에서 호스트 네임플레이트가 보이지 않던 문제가 해결되었으며, 호스트와 클라이언트 양쪽 화면에서 정상 표시됩니다.

TROUBLESHOOTING 01

호스트 네임플레이트 — 복제가 아니라 로컬 가시성 문제

오류 현상 · 원인 파악

- 클라이언트 화면에서는 호스트 이름이 보임
- 호스트 자신의 화면에서만 호스트 이름이 숨겨짐
- 따라서 GameState 이름 복제 자체는 동작
- 현재 함수가 모든 로컬 제어 Pawn을 명시적으로 숨김

호스트만 보이는 이유

Listen Server 호스트는 Authority이면서
`IsLocallyControlled()==true`

원인 파악



로컬 가시성 처리 정리



양쪽 화면 정상 표시

기존 원인 코드 · 해결 결과

```
if (IsLocallyControlled())
{
    NameplateWidgetComp→SetVisibility(false);
    NameplateWidgetComp→SetHiddenInGame(true);
    return;
}
```

해결 결과

로컬 캐릭터도 표시 대상에 포함되어 호스트와 클라이언트 화면에서 네임플레이트가 정상 표시됨

해결 확인

Host→Host 정상

Host↔Client 정상

이름 복제 경로 유지

해결 완료

TROUBLESHOOTING 02

C++ UFUNCTION과 Blueprint 노드 버전이 어긋난 문제

오류 현상 · 원인

- WBP_SkinCustomization 로드 시 기존 Apply Skin / Equip Skin 노드 핀을 찾지 못함
- 현재 C++에는 두 함수가 BlueprintCallable로 존재
- Editor DLL은 7월 22일, 관련 헤더는 7월 23일 수정본
- 즉 Blueprint가 읽는 Reflection 정보가 현재 소스보다 오래됨

원인

소스 · 생성 코드 · 로드된 Editor 모듈의 버전 불일치

Editor 종료



Parcel_KnightEditor 빌드



Refresh All Nodes



Compile / Save

검증 · 해결 기준

```
// 현재 소스
UFUNCTION(BlueprintCallable)
void EquipSkin(FGameplayTag SkinTag);

UFUNCTION(BlueprintCallable)
void ApplySkin(FGameplayTag SkinTag);
```

완료 조건

에디터 재시작 뒤 두 노드의 Target·Skin Tag 핀이 복구되고 Blueprint 컴파일 오류가 없어야 함

확인 결과

UE 5.5 읽기 전용 로드에서 불일치 재현

원본 에셋은 수정하지 않음

[Blueprint 노드 캡처 및 실행 흐름 확인 필요]

TROUBLESHOOTING 03

DeliveryBox와 캐릭터의 물리 충돌이 일관되지 않던 문제

오류 현상 · 원인 파악

- 바닥 상자가 캐릭터를 통과하거나 반응이 일정하지 않음
- Root는 CollisionComponent, Mesh는 NoCollision
- 이전 생성자는 PhysicsBody 프리셋에만 의존
- Character Capsule의 PhysicsBody 응답도 코드에 없었음

검증

Git 594f37c 직전/현재 생성자와 Carry의 Attach/Detach 상태 비교

잘못된 설정 확인



Pawn ↔ PhysicsBody



Attach / Detach



Block 적용

해결 코드 · 개선 범위

- 상자: QueryAndPhysics + Pawn Block
- 캡슐: Pawn profile + PhysicsBody Block
- 운반 중: Actor collision·simulation 비활성
- 드롭 후: collision·simulation·hit notify 복원
- 던진 당사자는 0.5초 동안만 임시 무시

개선 결과

코드상 바닥/운반/드롭 상태가 분리됨. 실제 양쪽 화면 결과는 PIE 검증 필요

재검증

Listen Server + Client에서 밀기·들기·놓기·던지기

Zone overlap·배송 판정 회귀 확인

BP_DeliveryBox preset [에디터 확인 필요]

TECHNICAL ISSUE 01

서버와 클라이언트의 역할 분리

문제 상황

상태 위조 방지

점수·체력·배송 상태·Carry 상태를 클라이언트가 직접 바꾸면 모든 화면의 결과가 달라짐

TeamScore · HP · BoxStateTags · CarriedBox

어려웠던 상황

Listen Server

호스트는 서버와 로컬 플레이어 역할을 동시에 수행해 OnRep만 기다리면 로컬 표현이 빠질 수 있음

HasAuthority + IsLocallyControlled

판단 기준

판정과 표현 분리

게임 규칙은 서버, 지속 상태는 Replication, 소
유자 알림은 Client RPC, 순간 표현은 Multicast

intent → validate → mutate → replicate

최종 접근 · 서버

DeliveryZone, HealthComponent,
TeamScoreComponent가 Authority를 확인한 뒤
상태 변경.

최종 접근 · 상태

BoxStateTags, HP, Score, Inventory,
CarriedBox를 Replicated/RepNotify로 동기화.

최종 접근 · 표현

OnRep·delegate·Client/Multicast RPC로 HUD,
사운드, FX를 갱신.

TECHNICAL ISSUE 02

GameplayTags로 State와 Action을 분리

`Character.State.Carrying``Character.State.Ragdoll``Character.State.Dead``Character.Action.Interact``Character.Action.Throwing``Box.State.Held``Box.State.Delivered``Trap.Effect.Slow`

문 제 상 황

여러 Boolean과 Enum이 늘면 상태 조합과 시스템 간 전달 지점이 분산

어 려 율 던 상 황

Carry·Ragdoll·Dead 같은 상태가 입력, 이동, UI, 배송에 동시에 영향

판 단 기 준

지속 상태는 State, 입력/전이는 Action으로 이름 공간을 분리하고 Exact 비교 사용

01

정의

Config/Tags/*.ini에서 도메인별 태그 등록

02

저장·복제

UParcelPlayerStateComponent::CharacterStateTags를 RepNotify로 동기화

03

데이터 연결

FItemData, FBoxData, Trap DataAsset가 태그를 키로 사용

최종 접근

서버가 Add/Remove하고, OnRep delegate가 UI·애니메이션에 현재 컨테이너를 전달

TECHNICAL ISSUE 03

Lyra 스타일의 Component와 Interface 구조

AParcelCharacter

UParcelHeroComponent

UParcelInteractionComponent

UCharacterCarryComponent

UParcelMovementStatComponent

AParcelPlayerState

UInventoryComponent

UPlayerStatComponent

UCustomizationComponent

ADeliveryBox

IInteractableInterface

ICarryableInterface

UHealthComponent

AParcelGameMode

UDeliveryRuleComponent

URespawnComponent

문제 상황

Character·GameMode에 입력, 운반, 상태, 규칙이 집중되면 수정 범위가 커짐

어려운 점

Component 소유 Actor·초기화 순서·서버 RPC 위치와 Blueprint 연결을 함께 관리해야 함

판단 기준

수명주기는 소유 Actor, 기능 상태는 Component, 대상 능력은 Interface로 질의

최종 접근

Execute_CanInteract → Execute_Interact → Carry Component로 위임

정확한 표현

Lyra 공식 플러그인·코드 복제 근거는 없음. 책임 분리 “스타일”을 적용한 프로젝트 구조

TEAM RETROSPECTIVE

팀 프로젝트 회고와 Q&A

잘된 점

- 서버 연동과 멀티플레이 통신 방식의 이해 확장
- C++와 Blueprint를 함께 사용한 UI 제작 경험
- Actor 구조와 캐릭터 Enhanced Input 이해

아쉬운 점

- 기능 증가에 따른 코드 오염을 더 체계적으로 관리할 필요
- 병행 작업의 충돌·우선순위 관리 어려움
- .uasset 병합과 버전 관리의 한계
- 소통·일정·마일스톤 관리의 중요성 체감

발표 핵심 결론

서버 권한과 책임 분리를 기반으로
기존 배송 플레이를 유지하며
인벤토리·물리 상호작용을
확장했습니다.

배운 점

- 규모가 커질수록 브랜치 전략과 변경 범위 관리가 중요
- 파트 간 의존이나 충돌 가능성은 적절한 채널로 미리 공유

다음 개선

- 이번에는 멀티플레이 아키텍처와 기술적 완성도에 집중
- 다음에는 기본기를 다진 뒤 플레이어가 느끼는 ‘게임의 재미’까지 확장

Q&A

질문 감사합니다.